## Motivation

- Query and knowledge space mismatch
  - Vector retrieval assumes that the most semantically similar text to the query is also the most relevant. Which is not always true.
- Hard chunking breaks semantic and contextual integrity.
- Hard to deal with in-document reference
  - Like "see Appendix G" or "refer to Table 5.3".
  - Since these references don't share semantic similarity with the referenced content, traditional RAG misses them unless additional preprocessing.

## Indexing Flow

- There are **two retrieval levels**.
  - **Document-level index**: One record per document.
    "Which document should I search?"
  - **Document ToC tree**: One PageIndex tree inside each document.
    "Which section or page inside this document should I search?"

**Preparation:** Upload or register PDFs/documents before building the index.

1. **Create PageIndex ToC tree for each document**: Break each document into a hierarchical tree with metadata (node titles, summaries, page indexes, and raw content).
2. **Create document-level record**: Store `doc_id`, file name, metadata, title, description.
   **Note:** Can be derived from metadata of PageIndex tree. (e.g. root/top-level summaries).
3. **Optional document-search index**: Build SQL metadata search, vector search, or LLM-generated document descriptions for multi-document retrieval.

- If the document collection is small, it is usually better to create the **PageIndex ToC tree first**, then derive the document-level description from that tree.
- If the collection is very large, a cheaper document-level pre-filter can be built first, and full ToC trees can be generated only for likely useful documents.

## Production Query Flow

1. **Document search**: Use the document-level index to select relevant `doc_id` values.
2. **Tree search**: For each selected document, read its PageIndex ToC tree and select relevant `node_id` values.
3. **Retrieve raw content**: Use `node_map` to load node text, page images, tables, or other raw evidence.
4. **Answer the question**: Return a grounded answer with cited pages, nodes, or documents.

## Indexing Step 1: Create PageIndex Tree for Each Document Using LLM

- **Input**: Document
- **Output**: **JSON-based hierarchical structure** that represents **ToC**
- **LLM role**: The LLM can be used to identify section boundaries, generate titles, write summaries, and decide how detailed the tree should be.
- **Nodes** represent a section (e.g. chapter, paragraph, page)

```
Node {
  node_id: string,         // Unique node identifier
  title: string,            // Human-readable label or title
  description: string,     // Optional detailed explanation of the node
  metadata: object,        // Arbitrary key-value pairs for context or attributes
  sub_nodes: [Node]        // Array of child nodes (recursive structure)
}
```

  - `node_id`: Serves as a reference key to locate the corresponding raw data.
  - `sub_nodes`: Field allows recursive nesting, forming a full ToC tree.
  - `metadata`: Field can store semantic information (e.g. document type, author, timestamp, or relevance scores.)

- **ToC Example:**

```
...
{
  "node_id": "0006",
  "title": "Financial Stability",
  "start_index": 21,
  "end_index": 22,
  "summary": "The Federal Reserve ...",
  "sub_nodes": [
    {
      "node_id": "0007",
      "title": "Monitoring Financial Vulnerabilities",
      "start_index": 22,
      "end_index": 28,
      "summary": "The Federal Reserve's monitoring ..."
    },
    {
      "node_id": "0008",
      "title": "Domestic and International Cooperation and Coordination",
      "start_index": 28,
      "end_index": 31,
      "summary": "In 2023, the Federal Reserve collaborated ..."
    }
  ]
}
...
```

  - `start_index / end_index`: Start page / End page
  - Each node in ToC links to corresponding **raw content** (e.g. text, images, tables)

```
node_id → node_content (raw content, extracted text, images, etc.)
```

  - This mapping enables the LLM to **select and retrieve** specific nodes as needed.
  - **JSON-based ToC index** resides within the LLM's active reasoning context.

## Indexing Step 2: Create Documents Record Using LLM (Can be Same LLM)

The document-level record answers: "Which document should answer the query?" This record should be small, stable, and easy to filter.

- **Main fields**: `doc_id`, file name, title, source path, document type, author, date, language, and access permission.
- **Description**: A short document-level summary used for document search.
  - Create the PageIndex ToC tree first, then derive the document-level description from the top-level node summaries / metadata of PageIndex ToC tree.
- **Storage pointer**: The record should point to where the full ToC tree, raw text, page images, or original PDF are stored.

**Document-Level Record Example:**

```
{
  "doc_id": "fed_annual_2023",
  "file_name": "annual-report-2023.pdf",
  "title": "Federal Reserve Annual Report 2023",
  "description": "Annual report for document-level search.",
  "metadata": {
    "source": "Federal Reserve",
    "year": 2023,
    "document_type": "annual_report",
    "language": "en"
  },
  "toc_tree_uri": "storage/trees/fed_annual_2023.json",
  "raw_file_uri": "storage/files/annual-report-2023.pdf"
}
```

```
doc_id → document record → ToC tree / node_map / raw files
```

## Indexing Step 3: Build Optional Document-Search Index

This step is needed when the system has many documents. It helps choose which documents should be searched before running tree search inside them.

- **SQL metadata search**: Good for exact filters such as year, source, company, document type, or permission.
- **Vector search**: Good for semantic matching against document descriptions, titles, abstracts, or top-level summaries.

## Query Step 1: Document Search (Can be same LLM)

It uses the document-level index and returns candidate `doc_id` values.

- **Input**: User question, optional filters, and conversation context if needed.
- **Output**: A ranked list of candidate documents.

## Query Step 2: Tree Search (Can be same LLM)

The LLM reads the PageIndex ToC tree for each selected document, usually without the full raw document text, and selects relevant `node_id` values.

- **Input**: User question, selected `doc_id`, and that document's ToC tree.
- **Output**: Relevant `node_id` values, often with short reasons.

## Query Step 3: Retrieve Raw Content

After tree search selects nodes, the system uses `node_map` to fetch the real evidence.

## Query Step 4: Answer the Question (C)

- **Input**: User question, retrieved evidence, page numbers, `doc_id`, and `node_id`.
- **Output**: A grounded answer with citations to pages, nodes, or documents.

## Different Methods

### Document Search vs. Tree Search

In a multi-document system, retrieval is usually split into two levels.

- **Document search**: Find the relevant document records. The output is one or more `doc_id` values.
- **Tree search**: Search inside each selected document's PageIndex ToC tree. The output is one or more `node_id` values.

```
query → doc-search → doc_id → tree-search → node_id → raw content → answer
```

- **Document search tutorial** (`doc-search`): Shows how to choose documents using metadata, semantics, or LLM-generated descriptions.
- **Tree search tutorial** (`tree-search`): Shows how to choose relevant nodes inside one known document.

### Manual Text RAG: `demo/PageIndex_Simple`

This demo starts at the **tree-search** level because the document is already known.

1. **Convert PDF to Tree(ToC)**: PageIndex creates node titles, summaries, page indexes, raw text, and node ids.
2. **Tree search**: Give the LLM the question and the tree without raw text. The LLM returns relevant `node_id` values.
3. **Map nodes to content**: Use `node_map` to locate the full content for each selected id.
4. **Generate answer**: Feed only the selected node text to the LLM to answer.

**Production note:** For many documents, add a document-search step before this flow.

### Vision RAG: `demo/Vision_PageIndex`

This demo also starts from one known document, but the final answer is generated from rendered page images.

1. **Convert PDF to Tree(ToC)**: PageIndex creates node titles, summaries, page indexes, raw text, and node ids.
2. **Render pages**: Convert PDF pages into image files.
3. **Tree search**: Use a multimodal model as a text-only model to select nodes from the tree.
4. **Collect page images**: Convert selected nodes into page ranges and get page images.
5. **Generate visual answer**: Send the question and selected page images to a vision-language model.

**Production note:** Use this when figures, tables, scanned pages, or visual layout are important.

### Chat Quickstart: `demo/Chat_Quickstart_PageIndex`

This demo hides the tree-search and answer-generation details behind the PageIndex chat API.

1. **Submit document**: Upload a PDF and keep the returned `doc_id`.
2. **Check status**: Wait until PageIndex finishes processing.
3. **Ask question**: Call `pi_client.chat_completions(...)` with the user question and `doc_id`.
4. **Stream answer**: PageIndex handles retrieval and answer generation internally.

### Agentic Retrieval: `demo/Agentic_PageIndex`

Agentic retrieval is not just "LLM retrieves then LLM answers." Normal PageIndex can also do that. The difference is that retrieval becomes a **tool** inside a larger agent workflow.

1. **Submit document**: Upload a PDF and keep `doc_id`.
2. **Ask retrieval prompt**: Request raw relevant evidence in a structured JSON format.
3. **Parse JSON**: Extract page numbers and evidence content from the model output.
4. **Check / reuse evidence**: An agent can validate, compare, call more tools, retrieve again, or pass evidence to another step.
5. **Generate final response if needed**: The final answer may be produced after the evidence is collected.

**Key distinction:**

- **Normal PageIndex**: Retrieval is a step inside a fixed QA pipeline.
- **Agentic PageIndex**: Retrieval is an iterative tool that an agent can call conditionally.

### Comparison

| Method | Main Role | Flow Summary |
|---|---|---|
| Document Search | Select document(s) | Query → document index → `doc_id` |
| Tree Search | Select section(s) | `doc_id` → ToC tree → `node_id` |
| Manual Text RAG | Custom text QA | Tree → node ids → node text → answer |
| Vision RAG | Visual document QA | Tree → node ids → page images → VLM answer |
| Chat Quickstart | Simple hosted QA | Document id → PageIndex chat API → streamed answer |
| Agentic Retrieval | Structured evidence tool | Query → retrieval prompt → JSON evidence → agent use |